

Matching empirical datasets against thousands of intermediate symbolic atoms—especially those derived from raw Physics Derivation Graphs (PDGs)—is computationally prohibitive if your agents rely on iterative, non-linear least-squares optimization for every candidate.

Iterative curve fitting (like Levenberg-Marquardt) requires initial parameter guesses, gets stuck in local minima, and scales poorly. However, because Sciona's atoms are structured within a **Computational Derivation Graph (CDG)**, you possess deep algebraic lineage and graph topology. You can transition from **Optimization-Based Matching** to **Invariant and Signature-Based Screening**.

To scale this in Sciona, you can build a deterministic **Heuristic Funnel** that uses $O(1)$ metadata checks and $O(N)$ vectorized arithmetic to rapidly eliminate non-matches, reserving expensive numerical fits for the final handful of candidates.

Here are the most clever and efficient heuristics to implement in the Sciona framework:

1. CDG-Aware Equivalence Collapsing (The Search Space Reducer)

Many of the 1,000+ intermediate states in the PDG are mathematically identical, despite being structurally distinct steps generated by inference rules (e.g., $F = ma$, $a = F/m$, and $F - ma = 0$). To a data-fitting algorithm, these represent the exact same mathematical manifold.

- **The Implementation:** During Sciona's ingestion of `artifact_symbolic_expressions`, use SymPy to reduce every equation into a canonical zero-form: `sympy.simplify(LHS - RHS)`. Group the atoms into **Algebraic Equivalence Classes** based on this canonical hash.
- **The Heuristic:** The Sciona agent only tests the dataset against *one representative* per equivalence class.
- **Why it scales:** If the dataset fits the canonical form, it instantly matches every intermediate PDG step in that class. This pre-computation can collapse 1,000+ raw derivations into fewer than 100 mathematically unique surfaces to test.

2. "Invariant Variance" Fast-Pass (Parameter-Free Matching)

The primary reason least-squares is slow is the search for unknown physical constants (e.g., G , m , or k). You can bypass the optimization loop entirely by rewriting the intermediate expressions into **Conserved Invariants**.

- **The Implementation:** For an intermediate atom like $P = k \frac{T}{V}$, have SymPy rewrite it so all data-backed variables are on one side, and unknowns on the other: $k = \frac{P \cdot V}{T}$.
- **The Heuristic:** The agent computes the evaluated array of $\frac{P \cdot V}{T}$ as a highly-vectorized NumPy operation over the dataset rows. It then calculates the Coefficient of Variation ($CV = \frac{\sigma}{\mu}$) of the resulting array.
- **Why it scales:** If the physical relationship holds, the variance across the dataset will be near 0 (accounting for noise), and the mean μ is your fitted parameter k . You

replace an expensive iterative optimization with an $O(N)$ vectorized arithmetic pass.

3. Log-Space SVD for Multiplicative Signatures

Due to dimensional constraints (Buckingham π theorem), a massive percentage of unsimplified physical laws and intermediate states are monomials and power-laws (e.g., $F \propto r^{-2}$).

- **The Heuristic:** Take the natural logarithm of the positive dataset columns. Power laws transform into linear planes: $\log(y) = a\log(x_1) + b\log(x_2)$.
- **Execution:** Run a blazing-fast Singular Value Decomposition (SVD) on the log-transformed covariance matrix. The null-space vectors directly yield the integer/rational exponents present in the data (e.g., $[-2, 1, 1]$ for r, m_1, m_2).
- **Why it scales:** You aren't fitting equations; you are extracting the mathematical "fingerprint" from the data. The agent can use these extracted exponents to execute an $O(1)$ database query against the metadata of the artifact_symbolic_expressions to jump straight to the exact intermediate forms.

4. Graph-Directed Constraint Propagation

The true superpower of the Sciona architecture is the artifact_relationships and artifact_cdg_nodes. You can use the derivation operations to traverse the dataset intelligently.

- **Missing Variable Imputation:** Intermediate derivations often contain transient variables (like velocity v). If the dataset only contains position x and time t , standard matching fails. However, the agent can look at the CDG edge: if an upstream step is $\text{differentiate}(x, t)$, the agent can numerically differentiate the dataset (`np.gradient`) to synthesize a v column in-memory, unlocking the ability to test downstream intermediate atoms.
- **Topological Pruning:** Test the "root" premise equations of a CDG first. If a dataset fundamentally violates a root premise (e.g., Conservation of Energy), your agent can instantly assign a 0 applicability score to the entire downstream subgraph, pruning hundreds of intermediate states.

5. Multi-Fidelity RANSAC (Minimal Point Solvers)

When an intermediate atom *must* be fitted via least squares because it contains internal non-linear unknowns (e.g., $y = A \sin(\omega t + \phi)$) where log-linearization fails, do not run the solver on a 100,000-row dataset.

- **The Heuristic:** Use a RANSAC (Random Sample Consensus) approach. If the atom has K unknown constants, the agent samples exactly $K+1$ random rows from the data.
- **Execution:** It solves the exact algebraic system for those parameters. It then plugs those constants into the equation and evaluates the error against a tiny subset (e.g., 50 random points) of the remaining data.

- **Why it scales:** If an intermediate physical law accurately describes the data, it will hold true for any random subset. Full least-squares is only executed on the final $<1\%$ of candidates that survive this micro-RANSAC phase to establish the final `relative_fit_error`.

Summary: The Agent Workflow

By integrating these heuristics, an agent navigating your PDG-derived data follows this elegant cascade:

1. **Boundary Triage ($O(1)$):** Extract dataset properties (zero-crossings, monotonicity, available units) and drop incompatible equivalence classes.
2. **Exponent Extraction ($O(N)$):** Run Log-Space SVD to find the power-law fingerprint of the data.
3. **Invariant Variance ($O(N)$):** Run the vectorized `CV` check on the isolated parameters of the surviving candidate equations.
4. **Graph Simplification:** Once an intermediate atom is successfully matched, the agent traverses the `artifact_relationships` forward to collapse the intermediate state into the known, simplified physical law.